

BEATSYNC

Music-Reactive Arduino Lighting + Live Audio Dashboard

MASTER PROJECT CONTEXT / RESEARCH BRIEF

PROJECT MOTTO

Music is converted into data; data is converted into light. The same engine should work with a song first and a microphone later.

Purpose of this document: this is the shared source of truth for an 8-member team. Paste the “AI handoff prompt” near the end into any research AI before asking it for technical research. The AI should first understand the project architecture and unresolved engineering problem, then investigate only the assigned topic.

Item	Current decision / direction
Project type	Laptop web application + Arduino hardware lighting system
Core behavior	Song audio -> analysis -> lighting values -> Arduino -> physical LEDs
Website	HTML/CSS/JavaScript; dashboard is real UI, not a static mockup
Audio analysis	Web Audio API / FFT / waveform data; no AI required for core system
Connection	USB serial between laptop and Arduino; Web Serial API is the intended browser route
Prototype priority	Solve audio-to-LED mapping before investing in visual website design
Future mode	Microphone input using the same audio-analysis + lighting engine
LED scalability	User-configurable LED count; target prototype 8-20, possible expansion to 50+ addressable LEDs

1. Project history and evolution

The idea started as a simple disco-style experiment: when a song plays, LEDs on an Arduino kit should blink. The concept was then expanded into a web dashboard with lyrics, live music information, and a visual representation of the physical lights. The team then identified the real engineering problem: how to transform a changing audio spectrum into many independent LED behaviors so the lights do not all blink together.

The design evolved from a small number of ordinary LEDs toward a scalable lighting engine that can accept a configurable number of outputs. The future microphone mode was added so the system can react not only to a file being played by the website, but also to sound heard by the laptop microphone.

Stage	Idea reached	What it means technically
1	Disco LEDs	Arduino controls physical LEDs
2	Music-reactive LEDs	Laptop measures audio and derives control values
3	Dashboard	Website displays song, audio analysis and LED state
4	Scalable LED count	User configures how many LEDs are physically present
5	Advanced patterns	Different LEDs can have different intensity, phase, speed and color
6	Microphone mode	Same lighting engine accepts live microphone audio

What the team has already established

- The laptop/browser should be the main audio-analysis processor; the Arduino should primarily be the real-time physical light controller.
- The song itself is not electrically wired into an LED. Audio data is measured in software, converted into numeric control values, and sent over USB serial.
- Frequency and amplitude are different signals: frequency indicates where energy sits in the spectrum; amplitude/energy indicates how strong the sound is overall.
- The project does not need AI for the core behavior. FFT, thresholding, smoothing, mapping, timers, and animation rules are enough.
- The final interface should be dark/futuristic/galaxy-themed, but the hardware/audio prototype comes first.

2. The core engineering problem - the part to solve first

CORE QUESTION

Given a live audio signal and N physical LEDs, how do we calculate N different lighting states so the LEDs react musically, smoothly, and independently instead of blinking in the same way?

The target behavior

For example, with 20 LEDs, the desired system could produce states such as:

```
LED 01 -> intensity 35%, pulse 1.0 Hz
LED 02 -> intensity 70%, pulse 1.8 Hz
LED 03 -> intensity 18%, pulse 0.7 Hz
LED 04 -> intensity 92%, beat flash
...
LED 20 -> intensity 61%, traveling-wave behavior
```

These values must change continuously as the song changes. “Different” does not have to mean random: the behavior should come from the structure of the music and from the selected lighting algorithm.

Critical conceptual correction

A music frequency such as 100 Hz does not directly mean “blink an LED at 100 times per second.” For visual lighting, it is usually more useful to measure the energy in frequency ranges and use that energy as a control signal for brightness, pulse depth, movement speed, or pattern selection. Very high-rate electrical modulation and human-visible blinking are different concepts.

The first proof-of-concept should therefore be

1. Play or stream a known audio source on the laptop.
2. Read its time-domain waveform and frequency-domain spectrum.
3. Group the spectrum into a configurable number of logical bands/features.
4. Normalize each feature to an output range such as 0-255.
5. Map those values into N LED states.
6. Send a compact message over USB serial.
7. Have Arduino render the received states on real LEDs.
8. Compare the physical result with the website visualization and refine smoothing/mapping.

3. Audio analysis: how a song becomes numbers

3.1 Audio samples

Digital audio is a time sequence of numerical samples. The browser already has access to these samples inside the Web Audio processing graph. The project does not need to “guess” the song frequency; it measures the current audio signal.

3.2 FFT

An FFT converts a short time window of the audio signal into frequency-domain information. The Web Audio API exposes this through `AnalyserNode`. Its frequency-domain byte array contains values from 0 to 255, with bins spread from 0 Hz to one-half of the audio sample rate. For a 48 kHz sample rate, the top of the analyzed range is 24 kHz.

A useful approximation for the center frequency represented by a bin is:

$$\text{frequency_hz} = \text{bin_index} * \text{sample_rate} / \text{fft_size}$$

Example: sample rate 48,000 Hz and FFT size 1,024 gives a bin spacing of about 46.875 Hz. The team should verify the exact audio context sample rate at runtime rather than hard-code assumptions.

3.3 Waveform / amplitude

The analyser can also expose time-domain waveform data. The project can derive a loudness-like control value from the waveform, for example using RMS energy. This is different from the frequency spectrum. The analyser also supports a `smoothingTimeConstant` so displayed/control values can change less abruptly.

3.4 Example data

Raw FFT bins (simplified)
[12, 18, 25, 43, 61, 72, 91, 82, 65, 54, ...]

After band grouping
Bass = 215
Low-mid = 144
Mid = 91
High-mid = 173
Treble = 201

After normalization / mapping
[215, 144, 91, 173, 201, ...]

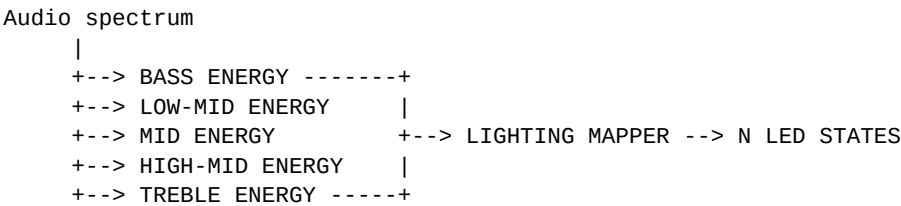
IMPORTANT

The exact mapping is an engineering design choice. The team should experiment with linear bands, logarithmic bands, smoothed bands, and feature-based mappings before finalizing the LED algorithm.

4. How frequency becomes many independent LED behaviors

4.1 Logical bands vs physical LEDs

The number of physical LEDs does not have to equal the number of frequency bins. A lighting mapper sits between the audio analyzer and the LED array. It converts many raw FFT bins into a more manageable set of musical features and then distributes those features across the available LEDs.



N = user-configured physical LED count

4.2 Three useful mapping strategies to research

Strategy	Idea	Advantages	Risk / issue
Band mapping	Partition spectrum into fixed ranges; each LED/group follows a range.	Simple, explainable, easy to debug.	May feel static with complex songs.
Spectrum-to-position	Treat the LED strip as a visual spectrum; energy at each range creates brightness/height.	Looks like an equalizer; highly intuitive.	Needs smoothing and careful scaling.
Hybrid feature mapping	Combine band energy + overall amplitude + beat events + time-based animation.	Most “concert-like” and visually rich.	More parameters and tuning.

4.3 Example with 20 LEDs

One practical prototype is to divide the LED array into five musical zones while still keeping individual control inside each zone:

```
LED 01-04 -> Bass response
LED 05-08 -> Low-mid response
LED 09-12 -> Mid response
LED 13-16 -> High-mid response
LED 17-20 -> Treble response
```

Inside each zone, neighboring LEDs can receive slightly different intensities or phase offsets. This is what prevents the whole zone from becoming one identical blinking block.

4.4 Brightness formula - starting point

```
band_energy    = average(FFT bins inside band)
amplitude      = smoothed overall signal strength
base_brightness = normalize(band_energy)
output         = base_brightness * amplitude
output         = clamp(output, 0, 255)
```

5. Beat detection and “dancing” behavior

5.1 Beat is an event, not a frequency

A beat detector can be built without AI. A simple starting approach is to compare current short-term energy with a moving average of recent energy. A significant positive jump can trigger a beat event.

```
current_energy = recent_audio_energy()
reference      = moving_average(history)

if current_energy > reference * beat_threshold:
    beat_detected = true
```

The threshold and refractory period must be tuned to avoid double-triggering from the same drum hit or triggering on every loud syllable.

5.2 Use time-based animation, not blocking delays

For independent LED pulse speeds, the controller can maintain a phase/time value per LED or per effect. This allows LED 1 to pulse differently from LED 2 without blocking the rest of the system. Avoid long delay() calls in the final Arduino loop because they reduce responsiveness.

```
for each LED i:
    phase[i] += delta_time * speed[i]
    brightness[i] = base[i] * pulseFunction(phase[i])
```

5.3 Good “concert-like” patterns

- Pulse: all or selected LEDs breathe in/out with audio energy.
- Wave: a bright region travels across the LED array; speed is controlled by tempo/energy.
- Spectrum: LED groups represent low/mid/high audio energy.
- Beat flash: a short high-intensity event is triggered by detected beats.
- Chase/comet: a brighter head travels with a fading tail.
- Hybrid: spectrum controls brightness while beat events trigger transient flashes.

6. Hardware architecture

6.1 Ordinary LEDs - useful for the first electrical test

A normal single-color LED is simple: Arduino output -> resistor -> LED -> ground. Brightness can be controlled by PWM on PWM-capable pins. On an Arduino UNO R3, the official documentation lists six PWM outputs (D3, D5, D6, D9, D10, D11). [Ncited0turn961706search0Öturn961706search2Ö](#)

LIMITATION FOR 20+ INDEPENDENT BRIGHTNESS CHANNELS

A typical UNO R3 does not provide 20 independent hardware PWM channels. Therefore, 20 or 50 ordinary LEDs need additional driver hardware or a different architecture. Do not plan to connect 20-50 LEDs directly to 20-50 Arduino pins.

6.2 Addressable RGB LEDs - preferred final architecture

For the scalable version, the team should investigate addressable RGB pixels such as WS2812B/NeoPixel-style devices. A single data signal can carry color/brightness data for a whole chain, while the individual pixels retain their own state. This is a much better fit for a 20-50 pixel “dancing light” installation.

Power must be designed separately from the data connection. Adafruit’s NeoPixel guidance notes that a single pixel can draw up to about 60 mA at maximum white brightness, so 50 pixels could theoretically approach 3 A at that extreme. A 5 V external supply sized for the expected load is therefore part of the final design, along with common ground. Adafruit also recommends a 300-500 ohm data-line resistor near the first pixel and a 500-1000 uF capacitor across the supply in typical setups.

[Ncited0turn322448search3Öturn322448search8Ö](#)

6.3 Hardware recommendation for the project

Phase	Recommended hardware	Purpose
Electrical proof	Existing red + green LEDs, plus a few ordinary LEDs and resistors	Verify Arduino control and PWM basics
Audio-to-light MVP	8-20 independently controllable outputs	Prove frequency/energy mapping
Scalable demo	20-50 addressable RGB pixels	Create rich independent colors/patterns
Future live mode	Same LEDs + laptop microphone	Reuse the same lighting engine

7. Website-to-Arduino communication

7.1 Why USB serial

The laptop needs a reliable way to send the calculated LED states to the microcontroller. The intended path is USB serial. The Web Serial API gives a website a way to read from and write to serial devices that expose a serial port, with browser security/permission requirements. The writable stream can send encoded bytes to the connected port. `Ëturn322448search10Ëturn322448search10`

7.2 Suggested message format

Start with a human-readable protocol because it is easy to inspect in a serial monitor. Later, the team can move to a binary packet if bandwidth becomes important.

```
CONFIG,20
FRAME,180,142,90,173,201,88,44,167,120,95,130,76,203,188,54,77,160,104,199,61
BEAT,1
MODE,HYBRID
```

The final frame format should include a sequence number or frame timestamp if the team later needs diagnostics and latency measurements.

7.3 Data flow

```
Browser
-> audio analyzer
-> lighting mapper
-> LED frame [N values]
-> serial encoder
-> USB
-> Arduino serial parser
-> LED driver / animation engine
-> physical LEDs
```

7.4 Target update rate

Do not send every raw FFT bin every animation frame. A much smaller control frame is sufficient. A practical starting target is around 20-50 lighting updates per second, subject to testing. The team should measure serial throughput, browser CPU load, and visual responsiveness before fixing the final rate.

8. User-configurable LED count

Yes - the website should ask how many LEDs are physically connected. This is not just a cosmetic setting; it changes how the lighting mapper distributes control values.

```
User input
LED count = 20
LED type  = addressable RGB
layout    = linear
mode      = hybrid
```

Lighting engine allocates N = 20 output states.

Recommended settings panel

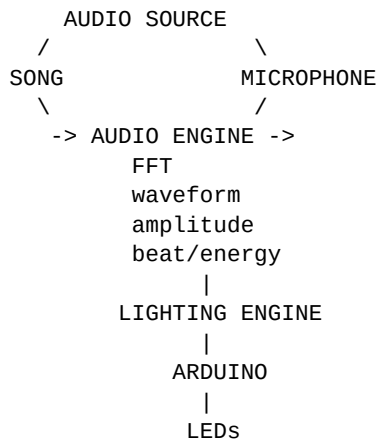
Setting	Example	Why it matters
LED count	20	Determines output array size N
LED type	WS2812B / individual PWM LED	Determines driver method
Arrangement	Linear / circle / matrix	Affects visual mapping
Lighting mode	Spectrum / pulse / wave / hybrid	Selects behavior
Sensitivity	65%	Controls response strength
Smoothing	0.75	Controls how quickly values react
Brightness cap	80%	Controls power and visual comfort

Scalability principle

The lighting engine should work with an array of length N. Avoid hard-coding “8 LEDs” deep inside the code. The same software should be able to run the algorithm for 8, 20, or 50 LEDs, with the hardware layer responsible for the physical output method.

9. Future expansion: microphone mode

The microphone should be added only after song mode is stable. The key architectural decision is to keep the audio analyzer independent from the audio source.



For microphone mode, the browser can request microphone access with `getUserMedia()` and feed the resulting `MediaStream` into the Web Audio processing graph. The same analyser/mapping code can then be reused. The team should test permission handling, echo/feedback behavior, and microphone gain.

Two operating modes

Mode	Input	Song metadata / lyrics	Lighting
Song Mode	Audio element / uploaded song	Available	Uses song timeline + audio analysis
Microphone Mode	Live microphone stream	Not required	Uses only live audio analysis

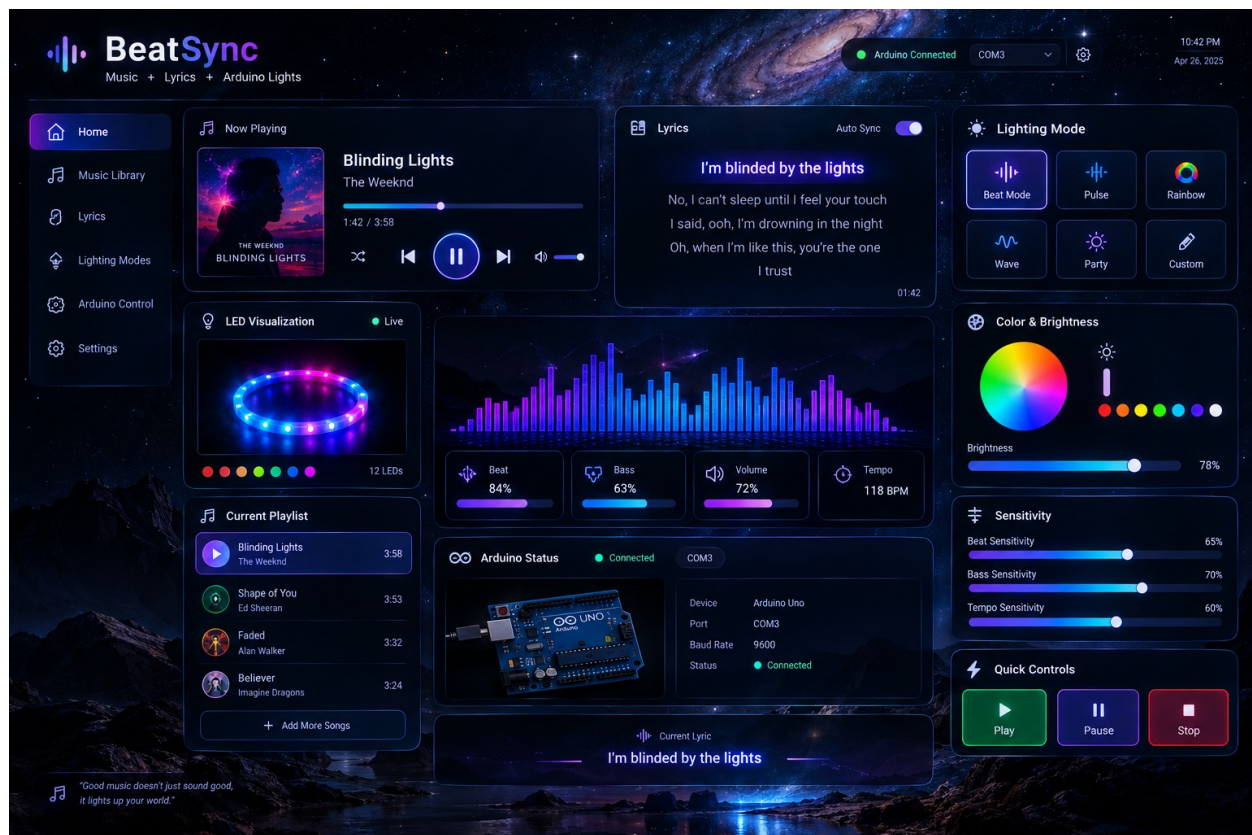
10. Website concept - after the hardware MVP works

10.1 Dashboard features already defined

- Current song: title, writer/artist, artwork, progress, playback controls.
- Live waveform / amplitude visualization.
- Live frequency spectrum / FFT visualization.
- Bass / mid / treble / energy metrics.
- Beat detected indicator.
- LED visualization showing the calculated state of each physical LED.
- Lighting mode selector: beat, pulse, rainbow, wave, party, custom, hybrid.
- Color and brightness controls.
- Sensitivity / smoothing controls.
- Arduino connection status, selected COM port, baud rate, LED count and hardware type.
- Lyrics panel with timestamp-based synchronization (LRC is the recommended first version).
- Song playlist / library area.

10.2 Visual direction

The chosen visual direction is not an animated office/studio scene. It is a dark, futuristic, 3D-style galaxy / Milky Way background with deep blues and purples, subtle glow, and glass-like panels. The background should remain visually restrained so live charts and LED state are readable.



Reference: generated visual direction for the future dashboard. The final website should reproduce the structure/function, not use a static image as the interface.

11. AI vs non-AI: what the project actually needs

CORE SYSTEM: NO AI REQUIRED

The main pipeline is deterministic signal processing + rules. AI may be added later for optional features, but it should not be a dependency for the core demo.

Function	Non-AI method	AI needed?
Waveform	getBytesTimeDomainData / time-domain processing	No
Frequency spectrum	FFT via AnalyserNode	No
Amplitude / energy	RMS / band energy	No
Beat events	Energy onset / threshold + refractory logic	No
LED mapping	Rules, formulas, interpolation, smoothing	No
Patterns	State machines / timers / easing functions	No
Lyrics sync	Timestamped LRC parsing	No
Microphone mode	getUserMedia + same analyzer	No
Automatic lyrics generation	External service or AI could be used	Optional
Mood classification	ML model could be used	Optional

12. Execution plan - do NOT start with the website

TEAM RULE

The website is Phase 2/3. The first milestone is a bench test that proves audio -> numeric control -> Arduino -> independently changing LEDs.

9. Test one LED: ON/OFF from Arduino.
10. Test brightness on PWM-capable pins with a single LED.
11. Test multiple LEDs with different static brightness values.
12. Generate a test sine wave / tone sweep on the laptop and verify that the analyzer changes in a predictable way.
13. Write a tiny browser experiment that prints FFT arrays and band energies in the console.
14. Create an 8- or 20-output software lighting mapper and visualize the numbers without Arduino.
15. Connect the browser to Arduino over USB serial and send a fixed test frame.
16. Make Arduino render the received values.
17. Play real music and tune the mapping, smoothing, gain, beat threshold and animation.
18. Only after the above is stable, build the full dashboard around the verified engine.
19. Add lyrics and metadata.
20. Finally add microphone mode and reuse the same lighting engine.

Definition of “working MVP”

Test	Pass condition
Audio analysis	FFT / amplitude values visibly change when the song changes
Independent LED states	At least 8 LEDs can show visibly different values at the same time
Dynamic response	LED states change as music energy changes
Beat response	A detected onset can trigger a short flash or pulse
Serial link	No major dropped/corrupt frames during a normal test
Scalability	Changing N in configuration changes the output array correctly

13. 8-member team research division

The team should work in parallel, but all members use the same architecture and terminology. No member should independently redesign the overall project without recording the change in the project folder.

Member	Primary research task	Concrete deliverable
1 - Signal processing	FFT, sample rate, FFT size, bin mapping, frequency bands	Technical note + tested band-energy code
2 - LED mapping	Map M audio features -> N LEDs; linear/log bands; normalization	Mapping algorithm spec + examples
3 - Beat/energy	Onset detection, RMS/energy, thresholds, refractory period, smoothing	Beat detector prototype + test notes
4 - Hardware	Arduino pins, LED drivers, WS2812B options, resistors, power design	Wiring plan + component/BOM sheet
5 - Arduino firmware	Serial parser, frame structure, LED state engine, non-blocking timing	.ino firmware + protocol spec
6 - Web audio	Audio element -> AnalyserNode -> FFT/waveform pipeline	Standalone browser audio-analysis demo
7 - Web serial + integration	Web Serial permission/connection, send/receive, latency/error handling	Serial browser demo + integration test
8 - UX/website + microphone	Dashboard layout, LED visualizer, lyrics, future microphone mode	UI spec + microphone input experiment

Shared review rule

Every member must report: (1) what was tested, (2) hardware/software used, (3) exact input/output observed, (4) what failed, (5) what should be integrated, and (6) links to authoritative documentation. “It should work” is not a result; an observable test is.

14. Research questions that remain open

These are the important questions that the team should answer before freezing the architecture.

ID	Question	Why we need the answer
R1	What FFT size gives acceptable frequency resolution and visual responsiveness for the target laptop?	Controls bin spacing and update behavior.
R2	Should frequency bands be linear or logarithmic?	Human hearing and music energy are not uniform across frequency.
R3	How should band energy be normalized?	Prevents quiet songs from looking dead and loud songs from saturating.
R4	What smoothing/attack/release times look “musical”?	Prevents jitter and creates intentional fades.
R5	What beat/onset detector works reliably across genres?	A beat rule that works on one song may over-trigger on another.
R6	How many lighting updates per second are visually sufficient?	Balances latency, CPU and serial traffic.
R7	What is the best LED hardware for 20-50 independent channels?	Determines wiring, power and firmware complexity.
R8	How should N LEDs map onto a linear/circular/matrix physical layout?	The same audio data can look very different depending on geometry.
R9	How much end-to-end latency exists from audio -> browser -> serial -> LED?	Important for perceived synchronization.
R10	How should microphone gain, permission and echo be handled?	Needed for reliable future live mode.

15. Recommended project-folder structure

```

BeatSync/
|
+-- 00_Project_Context/
|   +-- BeatSync_Project_Master_Context_v1.pdf    <- THIS DOCUMENT
|   +-- team-decisions.md
|   +-- research-log.md
|
+-- 01_Hardware/
|   +-- wiring/
|   +-- BOM/
|   +-- test-results/
|
+-- 02_Audio_Research/
|   +-- fft/
|   +-- beat-detection/
|   +-- mapping/
|
+-- 03_Arduino/
|   +-- firmware/
|   +-- serial-protocol/
|
+-- 04_Web_Audio/
|   +-- prototypes/
|   +-- analyzers/
|
+-- 05_Web_Serial/
|   +-- prototypes/
|
+-- 06_Website/
|   +-- frontend/
|   +-- assets/
|   +-- lyrics/
|
+-- 07_Microphone/
|   +-- experiments/
|
+-- 08_Integration/
|   +-- test-plans/
|   +-- demo-build/

```

Recommended source-of-truth rule: decisions that change the architecture (LED technology, serial packet format, mapping model, update rate) must be recorded in team-decisions.md with date, rationale, test evidence and the member who proposed the change.

16. AI handoff prompt - paste this into a research AI

HOW TO USE

Give the AI this context first. Then ask the AI only about the specific research question. Tell it to distinguish verified facts, assumptions, and proposed experiments.

You are assisting an 8-member engineering team building "BeatSync".

PROJECT MOTTO:

Music is converted into data; data is converted into light.

CORE GOAL:

Build a laptop web application connected by USB serial to Arduino. The website plays a song, analyzes live audio using signal processing (FFT / waveform / amplitude / energy / beat events), and converts that analysis into N independent LED control states. The physical LEDs should react differently and smoothly, like a small concert/disco lighting rig. The website should show current song metadata, lyrics, waveform, frequency spectrum, audio metrics, and the live LED states.

IMPORTANT ARCHITECTURE:

Song or microphone -> Web Audio analysis -> Lighting Mapper -> USB Serial -> Arduino -> physical LEDs.

The browser/laptop is the audio-analysis processor. Arduino is primarily the physical light renderer. Core system requires NO AI.

SCALABILITY:

LED count N is user configurable (prototype 8-20; possible 50+ addressable RGB LEDs). The number of FFT bins is NOT the same thing as the number of LEDs. A lighting mapper must convert many frequency bins/features into N LED states.

CORE ENGINEERING PROBLEM TO SOLVE FIRST:

How do we calculate N different, musically meaningful LED intensity/pattern values from a live audio spectrum so that LEDs do not all blink together? Possible inputs to the mapper include band energy, amplitude, beat/onset, smoothing, timing phase, and user sensitivity.

HARDWARE DIRECTION:

A small ordinary-LED prototype is acceptable for early tests. For many independent color/intensity channels, investigate addressable RGB pixels such as WS2812B/NeoPixel-style LEDs and separate power design.

FUTURE MODE:

Add laptop microphone input later. Reuse the same audio-analysis and lighting engine; only the audio source changes.

RESEARCH RULES:

1. Separate verified facts from assumptions.
2. Prefer official documentation and primary sources.
3. Give equations, parameter choices, test procedures, and measurable criteria.
4. Do not jump to AI/ML unless the requested problem genuinely needs it.
5. Explain how the result integrates into the existing architecture.
6. Produce something the team can test on real hardware.

17. Technical references used for this brief

These references support the core platform facts in the brief. The project team should still re-check compatibility and board details for the exact hardware they own.

Reference	URL	Relevant point
MDN - AnalyserNode	https://developer.mozilla.org/en-US/docs/Web/API/AnalyserNode	Real-time frequency and time-domain analysis; FFT-based analyser behavior.
MDN - getByteFrequencyData()	https://developer.mozilla.org/en-US/docs/Web/API/AnalyserNode/getByteFrequencyData	Frequency-domain values in a Uint8Array scaled 0-255 and bin distribution.
MDN - Web Serial API	https://developer.mozilla.org/en-US/docs/Web/API/Web_Serial_API	Browser access to serial devices; permission and secure-context considerations.
MDN - SerialPort.writable	https://developer.mozilla.org/en-US/docs/Web/API/SerialPort/writable	Writing encoded bytes to a serial device from a browser.
Arduino - UNO R3	https://docs.arduino.cc/hardware/uno-rev3/	UNO R3 architecture and six PWM outputs.
Adafruit - NeoPixel Powering	https://learn.adafruit.com/adafruit-neopixel-uberguide/powering-neopixels	Power calculations, common-ground guidance, data resistor recommendation.
Adafruit - NeoPixel Best Practices	https://learn.adafruit.com/adafruit-neopixel-uberguide/best-practices	Supply capacitor and data-line protection guidance.

18. Master checklist before website development

Check	Status to reach
Arduino can control one LED	PASS / measured
Multiple LEDs can hold different states	PASS / measured
Brightness control strategy selected	Documented
Audio waveform visible in browser	PASS
FFT spectrum visible in browser	PASS
Band-energy numbers stable enough for control	PASS
Lighting mapper creates N independent values	PASS
Values transmitted over serial	PASS
Arduino renders the same values physically	PASS
Beat/onset detector has a tested rule	PASS
Smoothing and sensitivity controls tuned	PASS
LED hardware/power design approved	Documented
End-to-end latency measured	Recorded
Song mode is stable	PASS
Then - full website dashboard	START
Then - microphone mode	START after song mode

FINAL TEAM PRINCIPLE

Do not make the website look “finished” while the audio-to-LED algorithm is still unknown. First make the physical lights react correctly; then make the dashboard beautiful.